

Most metropolitan areas have testing and development user groups and meet-ups, which convene regularly. These can be a great resource for free training and information, as well as a place to meet people who have tried different tools and techniques. Make the most of these networking opportunities.

If you want to improve your teaching and coaching skills, consider a coaching retreat such as Agile Coach Camp. Agile games gatherings such as Play4Agile are a great place to get ideas for participatory learning and building trust in teams. Portia Tung (Tung, 2011) says that play breaks down barriers between people and opens the mind to enable learning.

There are workshops and courses designed to help you learn leadership and relationship skills, such as Problem Solving Leadership (Derby et al., 2014) and those offered by the Satir Global Network (Satir Global Network).

To get better at almost anything, we need to practice. Musicians, athletes, and video game players all spend time practicing. At code retreats (Haines et al.), programmers practice repeatedly writing code that they'll throw away. Lisa finds that code retreats are a great learning experience even if you're not much of a coder. She learned lessons about problem solving and practicing one's craft. There are good ways to practice testing skills, too; for example, attend a testing dojo, or organize your own.

Groups such as Weekend Testing give you the opportunity to practice testing techniques in real time with testers around your continent or around the world. If you have a spare hour and want to practice a specific skill, get someone to pair with you. Lisa recently reached out to her Twitter contacts, and several volunteered to pair with her virtually to practice Ruby Koans. Your own teammates may be ready and willing to help you practice.

You may be able to find courses offered in subjects that interest you. Many of these are on a specific topic such as user interface (UI) test automation. Some organizations bring instructors in to teach the whole team at once so everyone has a common understanding of the subject.

Online courses, screencasts, webinars, and tutorials offer convenient learning opportunities for all aspects of testing and agile development,

as well as the nontechnical thinking skills. Some online training is free or inexpensive, though more sophisticated courses, or those that include time online with the instructor, can cost more. You can usually work through them at your own pace.

Janet's Story

I recently took a course on personal kanban through www.udemy.com. I could take my time with it and work through the course anywhere I traveled and at my own pace; both aspects were really important to me. I used what I learned to help me focus on tasks that were highest priority and not get sidetracked on tasks that were less important at the current time.

If there's a specific skill you want to acquire, look for online videos from conference sessions, user group meetings, and training courses.

Publications, Podcasts, and Online Communities

Information about all aspects of software testing is available both in book form and on the Internet. We're fortunate to have many good books available about all aspects of software testing. You'll find references to books, articles, and blog posts throughout this book, and the bibliography for Part II has a complete list.

Podcasts are a convenient way to learn. You can listen to interviews with experts, training sessions on specific topics, conference keynotes, webinars, and panel discussions.

Podcasts Changed My Life

Steve Rogalsky, an agilist at *Protegra*, shares his story about podcast learning.

My wife likes to tell the story of when I came home from work and announced, "Podcasts have changed my life." Her natural response was: "What? Podcasts? Not me, not our children, but podcasts?" I persisted with my original assertion. "Yes, podcasts. They've changed my life."

Up until that time I had been largely relying on the organizations I worked for to do the hard work of identifying what I needed to learn next and how. When a friend commented several years ago that he listened to podcasts on the way to work, I decided to give it a try. After only a few episodes I was hooked. I looked forward to my time in the car when I could explore what other people were discovering about software development, testing, management, and other topics. On many occasions I would hear a new term, tool, or acronym in a meeting, download a related podcast before leaving work, and return the next day knowledgeable enough to apply the information at work.

I had discovered a way of learning that I enjoyed, that fit into my existing schedule, and that made my job more enjoyable. Through podcasts I was introduced to many new people and ideas that have made my work experience more successful and rewarding. Yes—by giving me a vehicle to own my personal learning, podcasts changed my life.

Look to the global testing and software development community for more people with whom you can exchange ideas and experiences. For example, the Agile Testing mailing list is a good place to ask if others have had the same problem you're facing and how they solved it. Online communities such as Software Testing Club are a great place to learn by participating in forum discussions and by blogging your own experiences. Mailing lists and social networks such as Twitter can introduce you to articles and blog posts on topics that interest you.

Open-source projects can be a good place to practice your skills, especially if you want some coding experience. You can refine other skills as you contribute to open-source projects, such as testing, writing help documentation, and providing training courses. (See the Part II bibliography section "Courses, Conferences, Online Communities, Podcasts" for links to the activities mentioned in this section.)

If testers are integrated into software delivery teams in your company, a testing community of practice (CoP) is a good place to learn and share experiences with others who are interested in testing. Lunch-and-learn sessions and book groups within an organization are other effective learning mechanisms. For example, several teams have read our previous book together, reading a different chapter each week. They spend a lunchtime or meeting discussing what they have read and how it applies

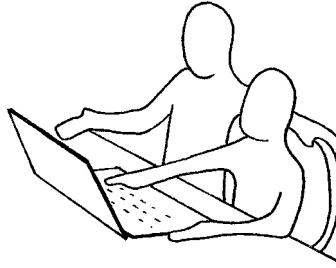


Figure 6-4 Pairing—a great way to learn

to their situation (Ruhland, 2013a). An added bonus of leading your team's journal club effort is honing your own facilitation skills!

Pair with your fellow team members (see Figure 6-4). Pair with people outside your development team. The marketing and sales people know a lot about your customers, as do your user experience (UX) designers. People in other departments such as human resources (HR) or accounting would provide a great fresh set of eyes on your product. There's a lot you can learn from your own coworkers, and a lot you can teach them.

Lisa's Story

I was part of a team that decided we would pair on every coding and testing task. When I paired with a programmer to automate UI acceptance tests that would guide coding, I noticed how naturally the programmer found duplication in the test code and immediately extracted it out into our library of macros. I sharpened my automated test design skills considerably as a result.

Iteration demos are another useful learning opportunity. Try rotating the duty of demonstrating what the team delivered. It's a great way to practice your presentation and facilitation skills.

TIME FOR LEARNING

Learning takes time. If you are constantly running just to keep up, you won't have time to learn and try new ideas. In Chapter 2, "The Importance of Organizational Culture," we talked about the significance of

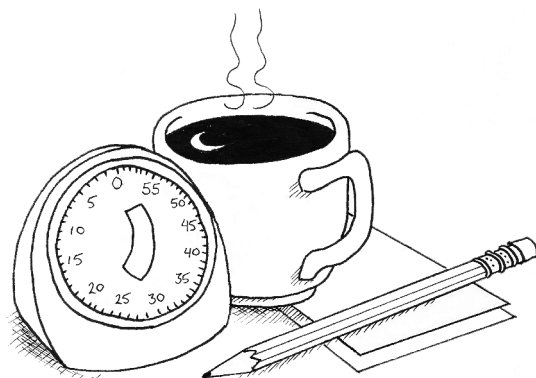


Figure 6-5 Make time for learning.

nurturing a learning culture so that team members feel they can take time to research, to experiment with new techniques, or just to think (see Figure 6-5).

Bernice Niel Ruhland suggests scheduling time for reading or other learning activities when you have the most energy. For example, if you're a morning person, try getting up a little earlier. Bernice allocates some lunchtimes and Sunday afternoons to reading.

Update Constantly

Mike Talks, a software tester from New Zealand, explains how learning is a constant challenge that all of us should be up for.

Most people's working life will span at least 40 years. When you look at the field of software development and work out how much change happened within that time, it's obvious that the skills of graduates today will feel obsolete come their retirement. Just looking back 10 or 20 years; it's like stepping into another world. Smartphones, tablets, broadband Internet—things we've already begun to take for granted—these are all recent developments.

When I started out 15 years ago as a programmer, I was told that FORTRAN and C would be the only languages I would need—and these have since been superseded by Java, C++, and C#.

What this means is that software professionals cannot just coast through their careers with their current skill set. New developments will mean new learning. Holding onto the mantra “We’ve always done it this way” is not good enough. One way to stay relevant is to find ways to continually learn and take on new ideas.

What I would really like to go on record as saying is that learning is itself an agile process. Don’t look at learning as a “big bang” process: you know nothing, read a book, and two days later you’re an expert. Learning, like features, is something you do in iterations, adding a bit more knowledge at a time, and then building on it the next iteration. It’s not a race to the finish, and there is always something more to learn. Often the people who seem to pick up knowledge more slowly are the ones who are actually learning it on a much deeper level.

In my book *The Software Minefield* (Talks, 2012), “Closing Thoughts” talks about channels of learning and is really like my experience report on learning.

If your team meets the “Agile Acid Test” (Hendrickson, 2010) and delivers software frequently at a sustainable pace, you should have some free time outside of work for professional growth. Just as musicians practice their instruments outside of performances, we all need to hone our skills outside of work. If you love what you do, this is a joy, not a burden. As Steve Rogalsky says, learning more will increase your passion and your joy.

HELPING OTHERS LEARN

All team members, including testers, can use their coaching and leadership skills to help their teammates learn.

Gaining Confidence

Aldo Rall, a test manager in South Africa, tells us how he helped new team members gain confidence.

New testers on our team were too afraid to participate in the normal collaborative mechanisms of working in agile teams. They also had

some cultural disadvantages that made them reticent to question or challenge the team's current thinking or process.

The organization was also not very well educated about what testing entailed. I implemented a lot of ongoing education for the clients, the business analysts, the programmers, and the project manager to open their minds to different ways of doing things and raise awareness of alternative approaches to development.

As a result of our training efforts, the testers started acting as a community of practice. Testers shared problems with each other and debated best solutions for their respective projects within this community of practice.

Loads more exploratory testing occurred. The testers were no longer afraid to speak their minds to their project teams. The testers were accepted as valued, contributing, cooperating, and collaborating team members. The programmers took advice from testers about releases into production seriously. Production incidents went down to a trickle. What is more, the testers developed a very important attribute: they had confidence in their ability to do testing.

Getting people over their fear of the unknown is a giant step toward empowering them to take control of their own professional development. As Aldo says, training and support for learning also help people reach out and collaborate with other team members.

Learning is often an important side effect of trying a series of small experiments to solve a problem that's holding up the team. What you learn may be more valuable than overcoming the original obstacle!



Surprise Learning

Claire Moss, a tester in the United States, shares her story of how she led her team through a series of small experiments to solve a problem. It generated unexpected opportunities for the team to improve communication and learn to collectively own testing activities.

As the tester on a cross-functional Scrum team, I knew about defects. Being a visual thinker, I put sticky notes on a small whiteboard by my

desk. Since I'd never used a board like this before, I tried a few different categories to better prepare for talking to my product team members about testing results. Although other people could see the board, I always thought of it as keeping things straight in my own head. Figure 6-6 shows my initial attempt.

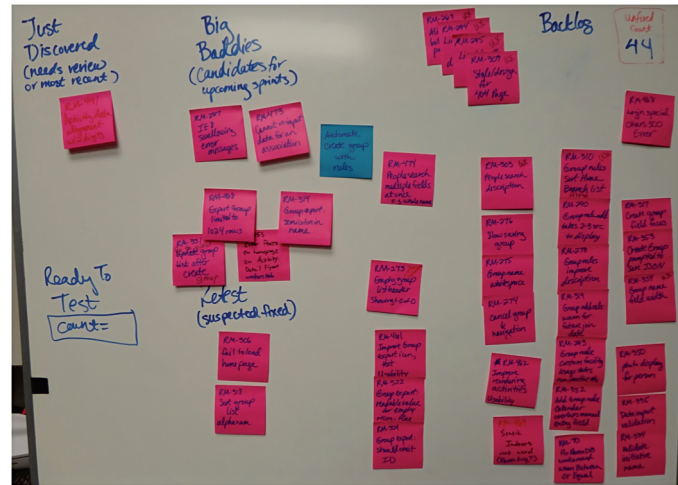


Figure 6-6 Initial bug board

One day, I walked into the office and found that my teammates had rearranged my sticky notes, engaging with my board. I was fascinated that they found my personal record keeping interesting and useful. By prioritizing my defect backlog, my product owner team revealed their desire for more information about testing. I seized the opportunity to use this big visible chart as a means to the end of better shared understanding.

I paid careful attention to the changes my colleagues made and did some research to see how other agile teams were using big visible charts. Thus began my series of experiments.

The most common question about defects seemed to be, “How bad is it?” so I tried rearranging the bugs based on severity (i.e., impact on the user). Since my developers scrupulously resolved the worst problems quickly, we ended up with large buckets of low-impact problems. Although our communication definitely improved, we still needed to represent the business’s judgment about priority. Not all bugs of a given severity are equally detrimental in the eyes of the business.

To satisfy this need, I tried another way of looking at the problem. Although I wanted to provide deeper information about the sources of bugs, initially I reported the user-facing symptoms. By co-opting the site map I'd made for test planning, I showed the bigger picture of where bugs were surfacing. The team could literally take a step back to see clusters of problems that might indicate a need for UX redesign or code refactoring. See Figure 6-7 for an example. That worked for a while as well.

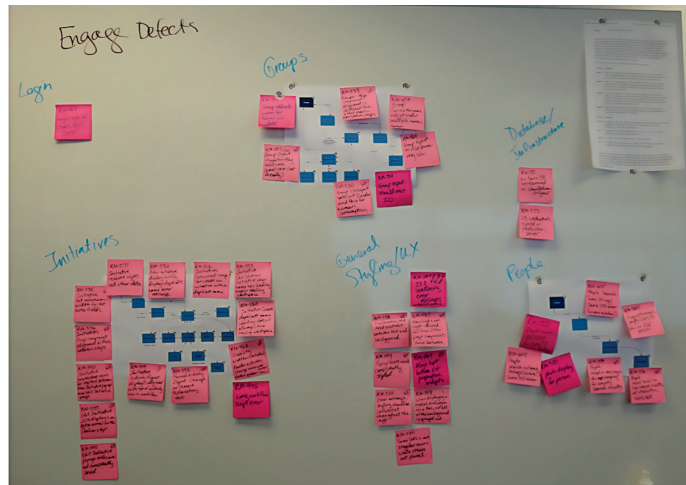


Figure 6-7 Bug board—clustered

All this time our team members were learning to talk more comfortably about each other's work. Each of us became more T-shaped, broadening our understanding of areas outside our specialization. We realized that the disconnection between the bug backlog and the user story backlog was a problem we had created for ourselves. So we removed the impediment.

We began pulling bugs into sprint backlogs based on priority, which included analysis of user impact.

We pulled related bugs into our stories to give us more context in which to execute the user stories we had planned. We disposed of the bugs during a story, either fixing them or delaying them as we deemed appropriate. Having this deliberate decision in place helped us to

focus on the whole product backlog so that we no longer needed or valued a separate bug board.

Representing all the team's development work as a single backlog was a step in the right direction. We adapted our big visible charts after conversations about the value they provided. However, all of these iterations focused on understanding defect reporting, which had been the most visible part of the testing activities. Our team was well on the way to a whole-team testing approach, but we still needed to delve deeper, beyond defects. Our big visible charts helped us to find the path and opened up team communication to encourage collective ownership of testing.

If you see someone you can help, take the initiative. Make a big visible chart as Claire did, start a testing community of practice, or perhaps a testing journal or book club. Sometimes, a new idea is as simple as seeing, and seizing, an opportunity.

SUMMARY

There's a lot to consider as you grow the T-shaped skills needed to succeed with testing. Widen your search to include less obvious opportunities, perhaps outside of the software development field.

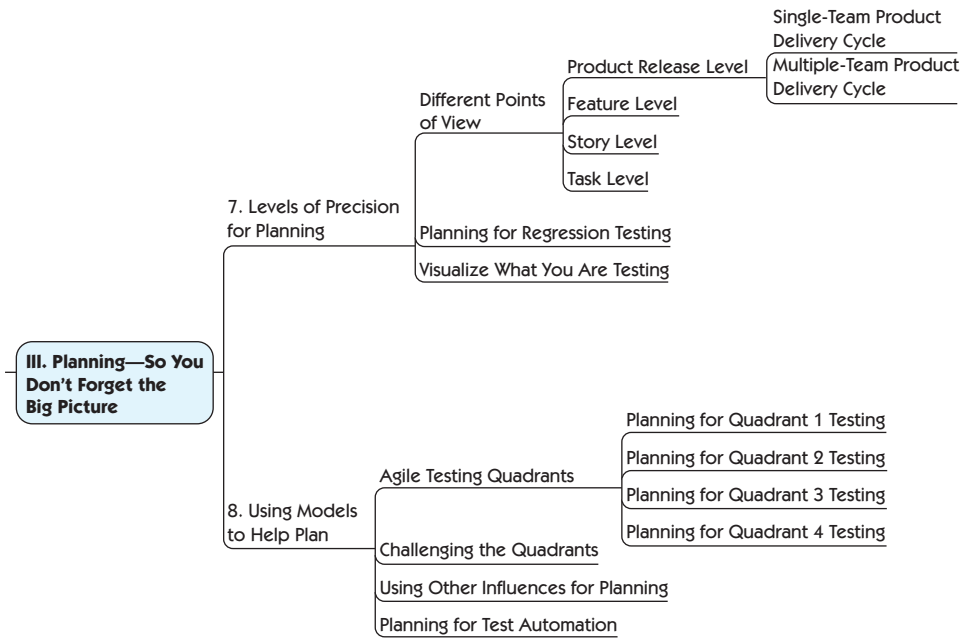
- Know your preferred learning style and use it as a basis to optimize learning opportunities.
- There is a wide variety of venues for learning. Try more than one and see which ones work for you.
- Build in time for learning.
- Try experiments to engage everyone on the team; work with business stakeholders and help them learn what they need to know.
- Keep in touch with the larger testing communities in local and Internet user groups, and then try the new ideas you get.
- Spend time practicing your skills.
- Take control of your own professional development.

This page intentionally left blank

PLANNING—SO YOU DON'T FORGET THE BIG PICTURE

It amazes us how many times we still hear, “But we don’t need to plan now because we are doing agile.” We agree that in agile development, we don’t do big, up-front planning all in one phase. But planning is important, and it is part of agile. We plan as we need to—just in time. Lean thinking refers to the idea of the last responsible moment, so that you don’t waste your time on details you don’t need yet.

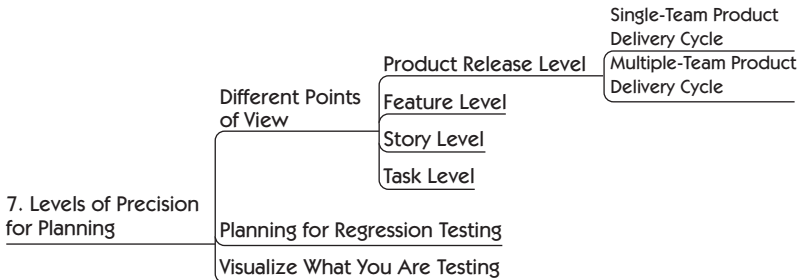
We divided this part into two chapters. In Chapter 7, we deal with the idea of levels of precision for planning: understanding what is needed at the level you are currently planning. In Chapter 8, we talk about how different models can help us plan our testing. We cover the agile testing quadrants, not in the detail that we did in *Agile Testing*, but to reiterate the concepts behind them. We introduce two variations on the Quadrants that express some of the changes in thinking over the last few years. We look at some of the other models that can help with planning, including the test automation pyramid.



- Chapter 7, “Levels of Precision for Planning”
- Chapter 8, “Using Models to Help Plan”

Chapter 7

LEVELS OF PRECISION FOR PLANNING



A frequent misperception about agile development is that it requires no planning, because the Agile Manifesto says agile practitioners value responding to change over following a plan. In reality, a good agile team tends to plan more than phased and gated project teams. It's done in smaller chunks, with enough for what is needed, and it uses fast feedback to learn and adapt. Test planning is the same—small chunks and short feedback loops. As you plan testing, focus on what you need to know now.

DIFFERENT POINTS OF VIEW

In small, colocated teams, planning for testing can be simple. The team considers the types of testing they need to do, perhaps using a model like the agile testing quadrants (see Chapter 8, “Using Models to Help Plan”) to help identify what testing needs to occur or what their automation requirements might be. Team members can vocalize their testing needs in planning sessions or represent them visually, perhaps on a whiteboard.

In large organizations where multiple teams work on the same product, test planning can get complicated. Regardless of company or project size, the basic principles apply.

In agile projects, we don't know all the details about each feature at the beginning of a delivery cycle. Some may be straightforward and predictable because we've done similar features before. Others may be more complicated, or they may be so new and unknown that it's difficult to come up with examples of desired behavior. Our approach must be different from one that assumes the requirements are known up front and where planning is based on that information. Using agile principles of simplicity and iterative development, we look at levels of planning (or levels of precision of detail) and ask what information the team and stakeholders need to know at each level. Different organizations will have different contexts, so you need to know what is important to your team and your organization. We discuss this approach in the context of time-boxed iterations, but the concepts can be applied to flow-based methods such as kanban. Teams using a flow-based process may release after each story is completed, or they may accumulate the stories until a feature is complete and then release to production.

Figure 7-1 shows four levels of detail that we have to consider when planning, while at the same time keeping the whole product (the system) in mind. We use the following terms for these different levels of precision or points of view for test planning:

- **Product release:** One or more teams working on a single product, releasing at specific intervals or on defined dates. A product release may have one or many features.
- **Feature:** Some business capability or piece of functionality that is useful to the business; it could be part of a larger feature set. A feature usually has many stories, and the entire feature may require multiple iterations to complete. A feature delivery team is the cross-functional team working on one or more features for a product stream. It may be the only team or perhaps one of many teams working toward a larger product release.
- **Story (may be referred to as a “user story”):** A small, testable chunk of functionality that generally can be finished (“done”)

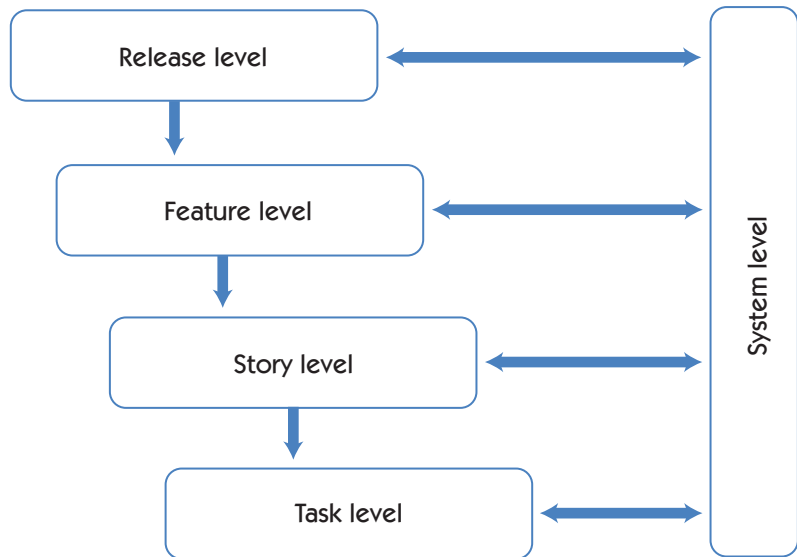


Figure 7-1 Levels of detail (precision) for planning

within one to three days. It may or may not be releasable to production on its own. A story has many tasks.

- **Task:** A piece of work that's part of a story, taking less than one day to complete.

For consistency of terms, let's assume we have a three-month (quarterly) product release with two-week iterations for each team. There are multiple features that each team completes in the release cycle. Figure 7-2 shows this setup.

Let's explore how you may adapt your test planning to each level, as well as discuss what documentation and/or artifacts might be expected at each of the four levels. At each level, we consider a different level of risk.

Product Release Level

At the product release level, teams should have a good idea of the product vision, perhaps by using impact mapping as discussed in Chapter 9,

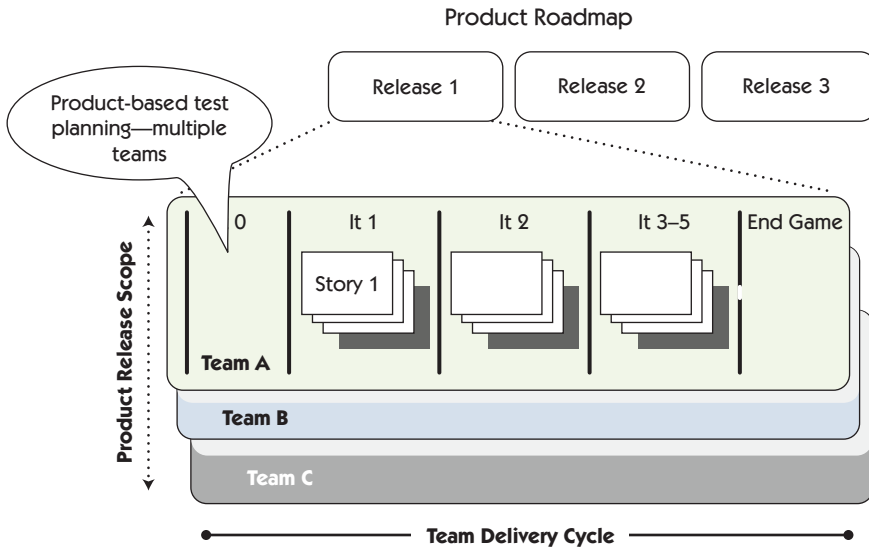


Figure 7-2 Levels of precision of planning for the product level

“Are We Building the Right Thing?” The high-level test approach should cover what is important to this product delivery cycle. The time when you are planning what will be in the release is the best time to identify the need for new test tools or environments.

This is also a good time to think about what additional testing will need to be done during the end game—the time before releasing a product to production. These activities may involve more groups outside your delivery team, such as operations, customer support, training, and marketing. For example, you may need to do final user acceptance testing (UAT) or test integration with an external system that couldn’t be done in the delivery team’s test environment. See pages 456–57 in *Agile Testing* and links in the bibliography for Part III for more information.



If the release includes new technology or features that are not yet well understood, the team might plan to do some spikes, small experiments to learn more about a possible problem or design solution. It may be appropriate to wait to learn from the spikes before trying to plan, even at a high level.

Single-Team Product Delivery Cycle

When your team is the only team responsible for delivering the release to your customers, your team is responsible for the completion of all testing activities. You may still need to use outside help such as performance experts, but your team is accountable. If yours is the only team working on a product, you can jump to the next section on feature-level test planning.

Multiple-Team Product Delivery Cycle

Having multiple teams work on the same product adds complexity, and an overall test approach should take into account dependencies between teams or between features. At this high level, there should be a few people who understand the big picture and how this release integrates into the system as a whole.

Janet finds that starting with a high-level context diagram (see Figure 18-2 for an example) helps visualize the risks and provides a starting point for thinking about dependencies outside the product teams or between features. By continually looking at the big picture, we can aim to find possible integration issues early and not only at the end of the delivery cycle. This will reduce the risk.

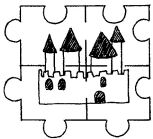
Consider the return on investment for tests such as interoperability, browser compatibility, or testing across different mobile devices. For example, individual teams may not be able to handle performance and load testing in their existing test environments. Consider the types of testing that might require a specialized test team, such as system integration. Chapter 18, “Agile Testing in the Enterprise,” presents ideas on how to handle testing in large agile organizations.

If it’s deemed useful to have a product release test plan written before you start your delivery cycle, we recommend keeping it as simple and clear as you can. It should not duplicate information in your organizational Test Strategy or Test Approach document (see Crispin and Gregory, 2009, p. 87). That document contains information that is common to all testing done for your product. For your current product release test planning, address only what is different for this particular release at a high level. Consider who the audience is and what is important for them to know. The less fluff there is, the more likely they will see the pertinent information about the potential risks in the new release.

At the product release level, test planning should include identifying testing risks and assumptions for the current release. It should highlight testing issues for possible cross-team dependencies as well as product integration issues.

Feature Level

As features representing some business-value capability are broken up into stories, we need to make sure they are testable so that the team gets into a cadence of coding and testing each individual story until the complete feature is “done.” Remember, a feature usually has many stories, and each delivery team concentrates on a particular feature. There may be multiple teams working in parallel, and there may be dependencies between the teams. Figure 7-3 shows how to think about dependencies between teams.



Each delivery team works with its product owner to determine how much work it might get completed during the delivery cycle. Often, a team release-planning session is held so teams can size the features and stories in their team product backlog. Testers help during this planning session by asking clarifying questions that may help the team determine the size of the feature or story. The team should consider impacts to the system as a whole.

Testers can also help when they apply an exploratory mindset to what performance or security concerns there might be, or whether a feature or story will affect other functionality. As with release-level planning, teams may defer test planning until after any necessary spike solutions or experiments are done and more is known about complex features.

Capture feature-level testing risks and assumptions that need to be monitored on a whiteboard, wiki, or other medium that is visible to both the delivery team and customers. Experiment with using diagrams, mind maps, bullet points, a matrix, or another lightweight format understandable to all. Watch how the teams use this information, and discuss it in retrospectives if you find there are communication gaps. Look for the best fit for your situation.

Sometimes clients or managers insist on a formal test document outlining what your team is going to test in this delivery cycle. Before you

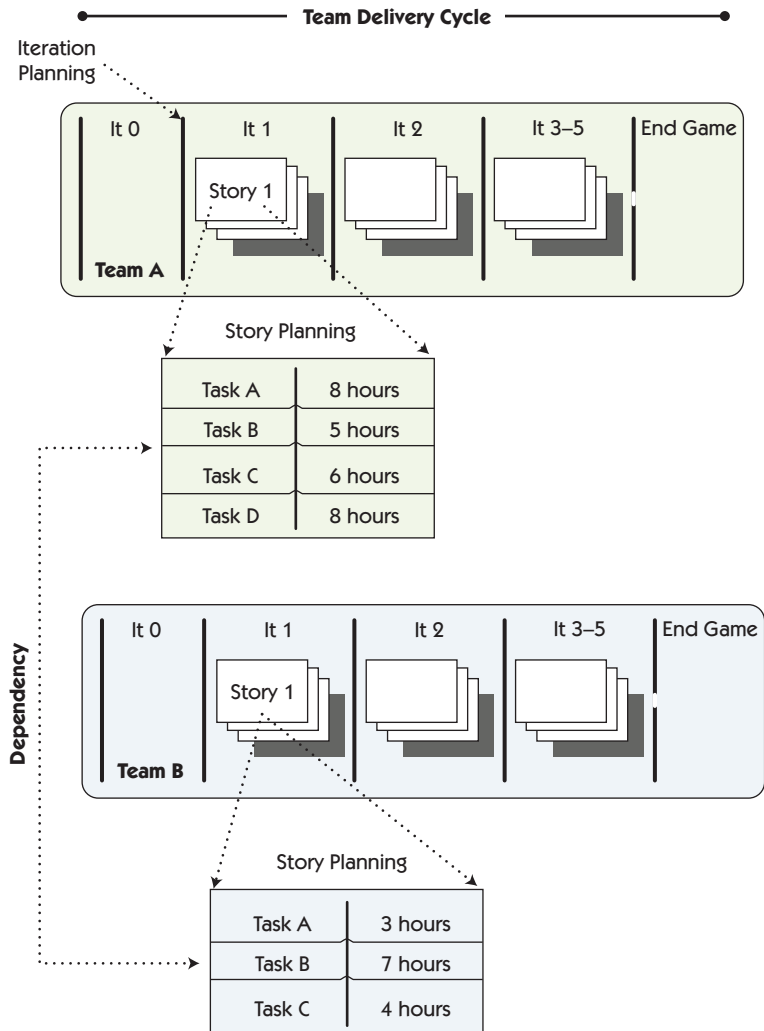


Figure 7-3 Levels of precision of planning at the team level

create a formal deliverable, think about who will use it and why they want it. Keep it as simple as possible—preferably less than one page. Since the functionality is not set yet, avoid getting into details of scope unless there is a risk involved that needs to be communicated to others. If a document is not needed, simply record the testing risks and

assumptions that need to be monitored on a whiteboard that is visible to the team.

At the 2011 Agile Testing Days conference, Huib Schoots told a story about an agile-resistant manager who demanded a document of the high-level feature and story tests. Instead of providing what was expected (a document), Huib created a mind map to capture those details. The mind map satisfied the manager and allowed Huib to keep to agile principles of simplicity, enabling him to stick to a one-page test plan to capture the important information.

Bernice Niel Ruhland is also a big fan of the one-page test plan (see Figure 7-4 for an example). She challenges herself to keep it to one page as an example to the testers, and as a practice manager, she created a basic template to get the testers started. They pick what they need and are free to add what they want, but she coaches them to find the right balance.

When it is time to break the feature into stories, work with your team's product owner to create high-level acceptance tests, using examples of expected behaviors and misbehaviors for the feature. Doing so will help define the scope and keep the business value visible. Mind maps are powerful tools for generating test ideas at any level, and we find creating them for each feature valuable. Try creating a testing mind map of the feature with your whole team before breaking it down into stories. It is one way to reveal and give an opportunity to explore some of the possible issues before they arise. Use an online mind-mapping tool if your team isn't colocated. Figure 7-5 shows an example of a testing mind map. See Chapter 9, "Are We Building the Right Thing?," for more details on ways to help customers define the most valuable features and slice them into smaller stories for development.

Planning testing activities for large features containing multiple stories that cross iterations can be complicated. Janet has found that teams benefit from creating a "Test the feature" story for these. The tasks in this story are mostly about testing activities, such as "Explore the feature," "Perform load testing," or "Automate the workflow." Lisa's team writes exploratory testing charters as stories in their online project-tracking tool. At the appropriate time, any team member or pair can perform these charters, or the feature-testing stories. This type of story ensures

Test Plan for Fitness Application Feature Adding BMI Calculator

Test Scope

- A BMI (body Mass Index) Calculator is being added to the fitness Application to calculate an individual's body fat. Data entered by the user is stored in the cloud and is shared across calculators with the ability for the user to change the data within each calculator. Initial testing will be performed using an iPhone, iPad mini, and a Nexus tablet.

Testing Approach

- The BMI calculator functionality will be tested based upon identifying the appropriate combinations and boundaries based upon the BMI requirements. Exploratory testing will be performed based upon how the users typically use the Fitness Application and anticipated usage of the new calculator. For this release, testing is limited to functionality, integration, exploratory, and regression. Device specific testing such as low-battery or other device conditions is not part of this testing.

Risks

- Look and feel across devices may not be consistent. Additional devices may need to be tested beyond the initial sample.

Regression

- Calculators using height and/or weight will be tested to ensure that shared fields across the calculators reflect the correct information and calculations are correct.

Features to Test

- BMI calculator (Functional testing)
- BMI Graph (Functional testing)
- Progress Report of Current Results Across Calculators (Integration testing)
- BMR Calculator & Body Fat Calculator (Regression testing)
- Cross-device testing based upon the usability of the screens

Features Not to Test

- Waist to Hip Ratio Calculator
- Weight Goal Tracker and Report
- Food Tracking Plan
- Settings Options

Data Validation

- Based upon US and Metric Scale using height and weight
- Identify combinations and boundaries (underweight, normal, overweight, and obese; and age range) to identify test data
- For other calculators using height and/or weight, identify previous tests to compare results when performing regression testing. Identify tests to perform across calculators to ensure the Progress Report of Current Results reflect the correct information

General Considerations

- Features not identified as part of the testing scope should have smoke tests performed before integration and exploratory testing begins.

Figure 7-4 Example lightweight test plan

that the feature testing is not forgotten. If your organization has an integration team doing post-development testing, they should be involved in defining these testing stories and tasks.

You may also want to think about developing personas and tours for extra exploratory testing ideas and coverage for each feature or for the end game. Chapter 12, “Exploratory Testing,” goes into more detail about these techniques.

Story Level

Once we are working at the story level, we start getting into more detail. At this level of precision, it doesn’t really matter whether you are using time-boxed iterations or flow-based methods. For each story, we need high-level acceptance tests: an example of expected behavior and at least one example of misbehavior that defines the scope of the story. Chapter 11, “Getting Examples,” gives more detail on this subject. Our just-in-time planning for the story level happens during story readiness sessions (sometimes called preplanning or backlog refinement).

Our tests for the story evolve from these sessions, as well as other conversations, and then can be expanded during the iteration. Start writing test ideas and variations during story readiness sessions, continue during iteration planning, and then use all the tools in your toolbox to define other tests that will prove that the story works as expected. Remember to write down exploratory test ideas through all of this, so the exploratory testing charters will be ready as code is completed. You’re likely to think of additional tests as you explore later.

Task Level

Programmers practice test-driven development (TDD), writing one test at a time before they code. This is a form of design, of planning what they are going to build at a detailed level.

When thinking about testing activities at the task level, do less planning and more doing and adapting. For example, if your task is to create test data, you might find yourself having to change where you get the data.

Some teams choose to estimate tasks in actual hours during their iteration planning. We find that this can be helpful for teams that are new to agile. The goal isn't to see how accurately you can estimate, but to learn where your time goes.

In *Agile Testing*, Chapter 17, we talked about several ways to create testing tasks, but we have found that creating separate task cards (virtual ones, if you're using an online tracking tool) for independent testing tasks seems to work the best. When a testing task takes significantly longer than anticipated, call attention to it during daily standups to ask for help or to see if it will affect the rest of the story. The team and the product owner should determine the best solution for finishing testing activities.

PLANNING FOR REGRESSION TESTING

Regression testing is about checking to make sure the system does what it did yesterday. Automated regression tests running in a continuous integration system deliver fast feedback to the team. As the team test-drives stories and features at the unit and business-facing levels, many (if not most) of the tests will be added to the regression suites. Your team needs to plan how to keep the suites up-to-date by determining what tests will be updated as you change or add functionality. When you remove functionality, you also must remember to delete the corresponding tests.

Regression tests need to be organized by system functionality so that you can locate tests easily to change, update, or delete them, or use them to demonstrate to someone how the system behaves. Many agile teams use an online tracking tool to maintain their stories, and some attach tests to the stories. This works when testing within an iteration but is not maintainable over the long run. Many teams find this out the hard way when they try to locate tests six months after the story is "done." Whether your tests are manual or automated, they should be migrated from a story-based grouping to an organization based on product functionality or business area.

Some teams add the story tests to the regression suite as soon as they complete a story; some wait until the feature is complete. Some teams

add their tests at the end of every iteration, deciding which ones provide enough valuable coverage to keep. Unit-level tests created as part of TDD are added automatically as they are checked into version control along with the code that they support. Your team should experiment with different ways of building and maintaining regression suites, evaluating their return on investment at regular intervals.

VISUALIZE WHAT YOU ARE TESTING

We find that visual aids help keep the whole team aware of what testing activities need to happen and track what still needs to be done. As mentioned earlier in this chapter, one effective way to visualize while brainstorming testing is mind mapping.

Start by putting your central theme or problem in the main “node,” and as you think of related topics, draw new nodes in an appropriate hierarchy. There’s no right or wrong way; just capture the ideas without criticizing them. Both whiteboards and online mind-mapping tools let you easily move nodes around as you go, and many online tools have a task completion marker to mark progress. Color coding is an effective way to show relationships. You might end up with a mind map such as the one in Figure 7-5.

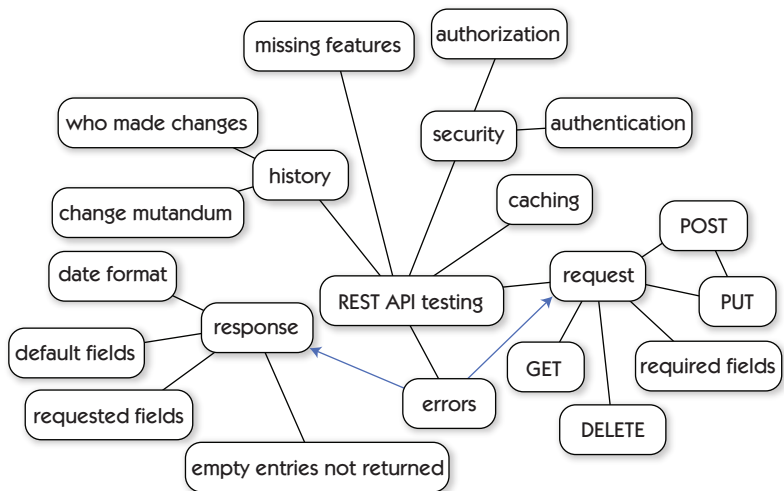


Figure 7-5 Mind map of testing ideas

A colocated team can keep the testing mind map up in the team area, have conversations around it, and update it, checking off the completed testing activities. Distributed teams can use an online mind-mapping tool, though it will take more practice and discipline to develop the habit and keep it virtually visible. A testing mind map keeps testing status front and center and reminds the whole team what testing remains to be done.

Another effective way to capture high-level test ideas at the release level is a test matrix. Collaborative techniques such as mind mapping can be used to generate test ideas for a release-level testing matrix, as shown in Figure 7-6. Visuals such as these provide a different viewpoint into planned testing activities. The rows are features, and the columns are test conditions/capabilities. In the example in Figure 7-6, the pattern shows progress: white means not tested; checkered (or green, if in color) means good to go; dots (yellow, if in color) means some testing done but may need more; striped (red, if in color) means broken; gray means not applicable, doesn't need testing.

Lots of Stuff Shopping Release 1.5	Data Integrity	Look & Feel	Calculations	Currency	Localization	Devices iPad	Devices iPhone	Data Integrity	Boundary Conditions	Load/Performance	Security
Store customer information											
Add to shopping cart											
Calculate shipping costs											
Mobile iOS-view only											
Next feature											

Legend	
Good to go	
Some testing; could use more	
Major issue	
No testing done	
Not applicable	

Figure 7-6 Release-level test matrix

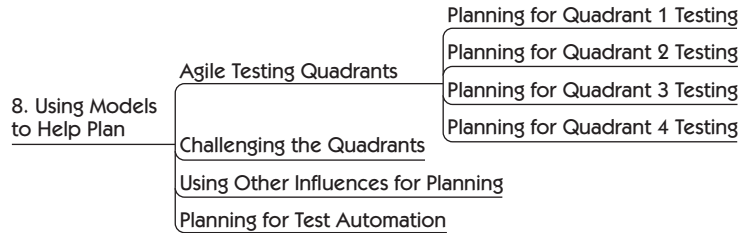
However you visualize your planned testing, try to keep it simple so that it can keep pace with your team. Use big visible charts or their virtual substitutes so all team members and stakeholders can see completed and planned testing at a glance. And make it valuable to your team or management; after all, you really want to be testing and not documenting.

SUMMARY

In this chapter, we talked about understanding what level of detail is needed for test planning as well as the importance of visualization.

- At the product release level, think about system impacts and what testing needs to be done during the end game, including, but not limited to, final UAT, load, and performance testing.
- If you are part of multiple teams working at the product release level, also consider high-level dependencies between components and teams, and what the integration system testing might include.
- For your delivery team planning at the release level, think about feature or story dependencies, risks, assumptions, and test environments.
- At the feature level, consider the impacts that affect this feature, the acceptance tests, and the types of testing needed, including quality attributes, usability testing, and user acceptance testing.
- At the story level, create acceptance tests, the remaining story tests, including exploratory testing, and maybe testing some of the quality attributes.
- At the task level, the planning tends to happen while you are performing the task such as TDD—writing the test first, then the code.
- Automated regression testing doesn't happen by chance. It needs to be planned and expanded as part of your everyday work.
- Find a simple way to make your testing visible, whether it is stored electronically or in a team room or hallway.

USING MODELS TO HELP PLAN



As agile development becomes increasingly mainstream, there are established techniques that experienced practitioners use to help plan testing activities in agile projects, although less experienced teams sometimes misunderstand or misuse these useful approaches. Also, the advances in test tools and frameworks have somewhat altered the original models that applied back in the early 2000s. Models help us view testing from different perspectives. Let's look at some foundations of agile test planning and how they are evolving.

AGILE TESTING QUADRANTS

The agile testing quadrants (the Quadrants) are based on a matrix Brian Marick developed in 2003 to describe types of tests used in Extreme Programming (XP) projects (Marick, 2003). We've found the Quadrants to be quite handy over the years as we plan at different levels of precision. Some people have misunderstood the purpose of the Quadrants. For example, they may see them as sequential activities instead of a taxonomy of testing types. Other people disagree about which testing activities belong in which quadrant and avoid using the Quadrants altogether. We'd like to clear up these misconceptions.

Figure 8-1 is the picture we currently use to explain this model. You'll notice we've changed some of the wording since we presented it in *Agile*

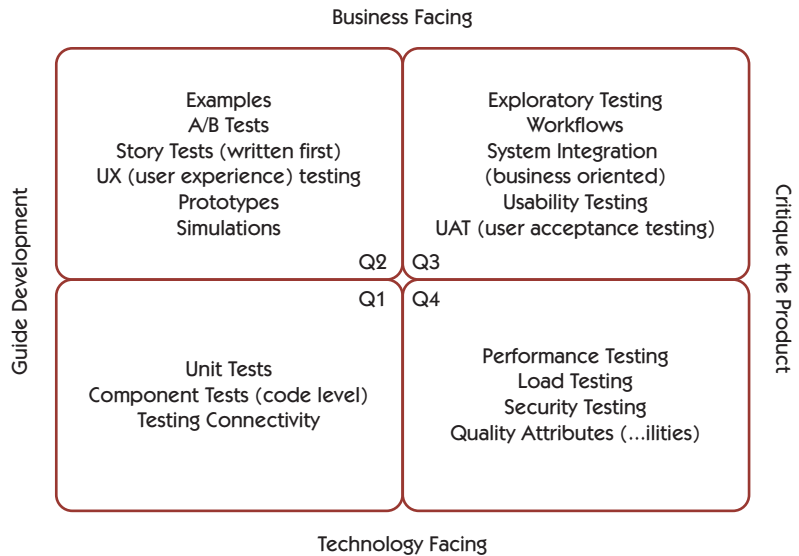


Figure 8-1 Agile testing quadrants

Testing. For example, we now say “guide development” instead of “support development.” We hope this makes it clearer.

It’s important to understand the purpose behind the Quadrants and the terminology used to convey their concepts. The quadrant numbering system does *not* imply any order. You don’t work through the quadrants from 1 to 4, in a sequential manner. It’s an arbitrary numbering system so that when we talk about the Quadrants, we can say “Q1” instead of “technology-facing tests that guide development.” The quadrants are

- **Q1:** technology-facing tests that guide development
- **Q2:** business-facing tests that guide development
- **Q3:** business-facing tests that critique (evaluate) the product
- **Q4:** technology-facing tests that critique (evaluate) the product

The left side of the quadrant matrix is about preventing defects before and during coding. The right side is about finding defects and discovering missing features, but with the understanding that we want to find them as fast as possible. The top half is about exposing tests to the

business, and the bottom half is about tests that are more internal to the team but equally important to the success of the software product. “Facing” simply refers to the language of the tests—for example, performance tests satisfy a business need, but the business would not be able to read the tests; they are concerned with the results.



Most agile teams would start with specifying Q2 tests, because those are where you get the examples that turn into specifications and tests that guide coding. In his 2003 blog posts about the matrix, Brian called Q2 and Q1 tests “checked examples.” He had originally called them “guiding” or “coaching” examples and credits Ward Cunningham for the adjective “checked.” Team members would construct an example of what the code needs to do, check that it doesn’t do it yet, make the code do it, and check that the example is now true (Marick, 2003). We include prototypes and simulations in Q2 because they are small experiments to help us understand an idea or concept.

In some cases it makes more sense to start testing for a new feature using tests from a different quadrant. Lisa has worked on projects where her team used performance tests for a spike for determination of the architecture, because that was the most important quality attribute for the feature. Those tests fall into Q4. If your customers are uncertain about their requirements, you might even do an investigation story and start with exploratory testing (Q3). Consider where the highest risk might be and where testing can add the most value.

Most teams concurrently use testing techniques from all of the quadrants, working in small increments. Write a test (or check) for a small chunk of a story, write the code, and once the test is passing, perhaps automate more tests for it. Once the tests (automated checks) are passing, use exploratory testing to see what was missed. Perform security or load testing, and then add the next small chunk and go through the whole process again.

Michael Hüttermann adds “outside-in, barrier-free, collaborative” to the middle of the quadrants (see Figure 8-2). He uses behavior-driven development (BDD) as an example of barrier-free testing. These tests are written in a natural, ubiquitous “given_when_then” language that’s accessible to customers as well as developers and invites conversation